

vectorised operations, and smart use of CPU resources.

For instance, DuckDB can filter and combine large DataFrames in milliseconds, while pandas can take seconds or minutes to do the same thing:

```
import duckdb
import pandas as pd
df = pd.read_csv('large_dataset.csv')
con = duckdb.connect()
con.register('df_view', df)

# Pandas work slowly
# result = df[df['sales'] > 1000].groupby('category').sum()
# Quick DuckDB query
result = con.execute('SELECT category, SUM(sales) FROM df_
view WHERE sales > 1000 GROUP BY category').fetchdf()
```

Using DuckDB's optimised query engine, avoiding Python-level loops, and allowing parallel execution, this SQL-based method often leads to big performance improvements—sometimes 10 times or more.

How to work with Parquet files

DuckDB and Parquet files work well together for fast data analysis. Parquet's columnar storage format is great for big data workloads because it has fast read speeds, compression, and efficient storage. DuckDB adds to this by having an embedded SQL engine that can query Parquet files directly without having to load or change the data.

One of the best things about using Parquet with DuckDB is that you can read more than one file at a time. DuckDB lets you treat a directory of Parquet files as a single table, making it easy to query large datasets that are spread out over many files. For example:

```
SELECT * FROM 'path/to/parquet_directory/*.parquet';
```

This method makes it easier to manage data and scales well, which makes it perfect for big data workflows. DuckDB also uses predicate pushdown and column pruning to speed things up. Predicate pushdown means that the filters in your SQL query are applied at the storage level, so only the data you need is read into memory. For example:

```
SELECT * FROM 'data.parquet' WHERE date >= '2023-01-01';
```

This filter is sent down to the Parquet file, which cuts down on I/O by skipping over data blocks that aren't needed.

Putting DuckDB into apps

DuckDB is great for embedded apps, Jupyter notebooks, and data science workflows where you need to quickly query and

change local datasets. It can easily save databases to disk, which lets you store and share processed data for a long time. This makes it useful for ETL pipelines, ML preprocessing, and interactive dashboards.

DuckDB speeds up data transformations, feature engineering, and preprocessing in ETL and ML workflows by running complicated SQL operations quickly, often better than pandas and SQLite. For dashboards, it gives quick query responses, allowing for real-time data exploration with little delay.

DuckDB is often faster than pandas for large-scale data operations, sometimes by a factor of ten, because it uses vectorized execution and an optimised query engine. It is also better for analytical workloads, like window functions and complex joins, than SQLite, making it a better choice for analytical applications.

DuckDB is great for big data analytics, interactive data exploration, and embedded data pipelines in the real world.

Things to think about for production

When deploying DuckDB in production, you need to think carefully about the amount of data, how many people will be using it at once, and what the operational needs are. DuckDB is good for analysing large datasets, but it might not work well with data that is very large—usually more than hundreds of gigabytes—where distributed systems like Spark or distributed SQL engines may be better. For workloads that are bigger than these sizes, you might want to split the data up or connect it to external storage solutions.

Concurrency and multi-user access are also important. DuckDB is best for environments with only one user or low concurrency; it doesn't support high levels of concurrent writes or multiple users at the same time, which is common in traditional client-server databases.

In conclusion, DuckDB has a lot of advantages: it's lightweight, easy to embed, fast for analytical workloads, and supports SQL directly on local files. It's best for embedded analytics, data science notebooks, ETL preprocessing, and prototyping. For large-scale, high-concurrency, or transactional systems, other solutions are better. Look at the specific workload needs, benchmark DuckDB in your environment, and add it to data pipelines to make the most of its strengths. **END** 

By: Sufyan bin Uzayr

The author, apart from being a writer, is an educator and entrepreneur. He holds multiple degrees in management, information technology and humanities. He has written several books and is currently the director of a multinational IT company. As an educator, he has taught at universities and institutions worldwide.